

Lab 12

Lab Title: Implementation of Layouts and Event Handling in Java (Swing)

Objective:

To understand and implement different types of **layout managers** and apply **event handling** using Java Swing to create real-world GUI applications.

Theory:

Layout Managers Overview:

Java Swing provides layout managers to control the arrangement of components inside containers:

1. **FlowLayout**
2. **BorderLayout**
3. **GridLayout**
4. **BoxLayout**
5. **CardLayout**

Each serves a different purpose and is used depending on the required user interface design.

1. FlowLayout

Theory:

Arranges components **horizontally**, left to right.

When there's no space, components move to the **next line**.

It's the **default layout manager** for JPanel.

Description:

Places components **horizontally** in the order they are added. If there's no space, moves them to the next line.

Real-world Scenario:

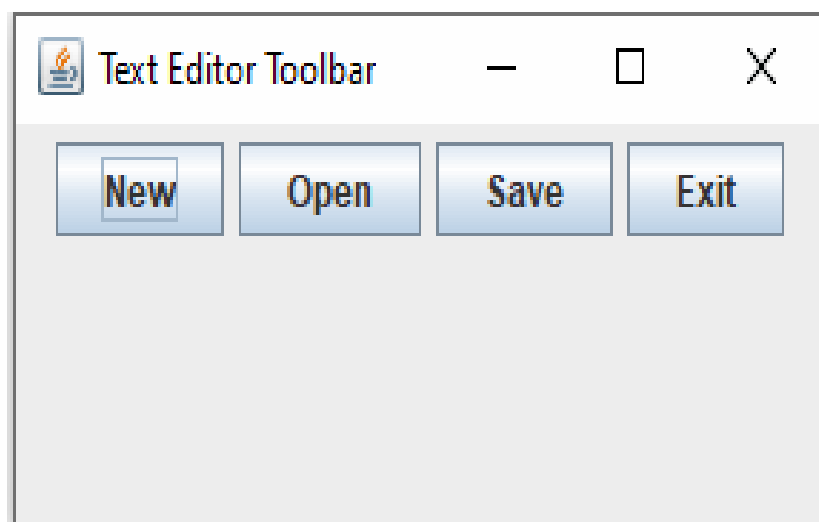
You are designing the toolbar section of a text editor where you place buttons like **New**, **Open**, **Save**, and **Exit** in a single horizontal line.

Code:

```
import javax.swing.*.*;
import java.awt.*.*;

public class FlowLayoutTextEditorToolbar {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Text Editor Toolbar");
        frame.setLayout(new FlowLayout());
        frame.add(new JButton("New"));
        frame.add(new JButton("Open"));
        frame.add(new JButton("Save"));
        frame.add(new JButton("Exit"));
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:



2. BorderLayout

Theory:

Divides the container into **five regions**: North, South, East, West, Center.
Each region can contain **only one component**

Real-world Scenario:

You are designing a **news app home screen** where:

The header goes to the **North**.

Footer with copyright goes to **South**.

Menu on the **West**, advertisements on the **East**.

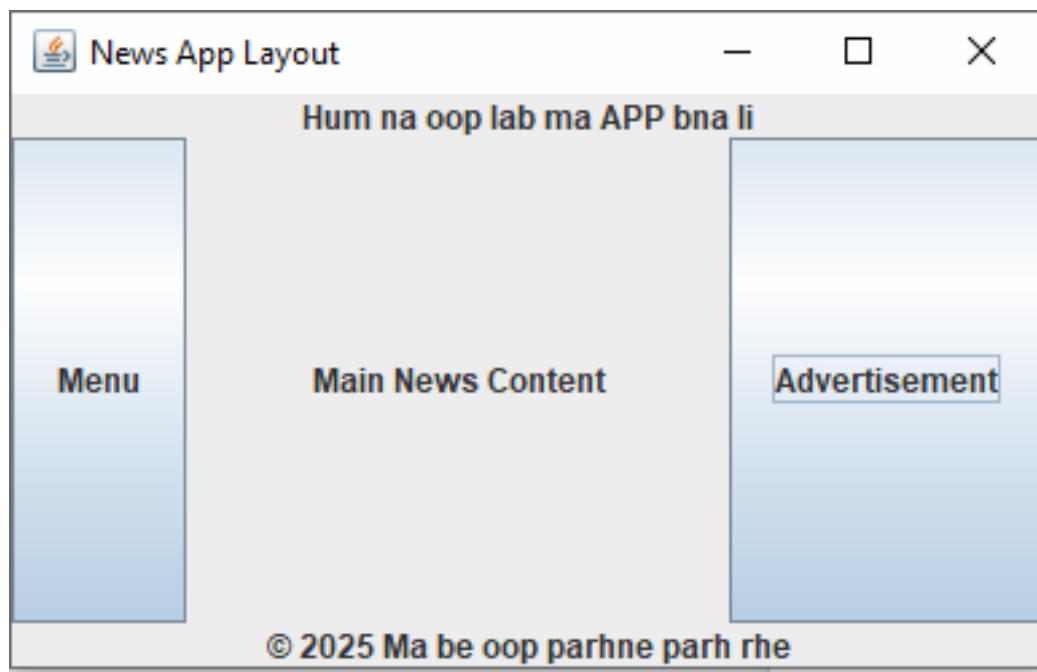
Main content in the **Center**.

Code:

```
import java.awt.BorderLayout;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
public class BorderLayoutNewsApp {
    public static void main(String[] args) {
        JFrame frame = new JFrame("News App Layout");
        frame.setLayout(new BorderLayout());
        frame.add(new JLabel("Hum na oop lab ma APP bna li",
JLabel.CENTER), BorderLayout.NORTH);
        frame.add(new JButton("Menu"), BorderLayout.WEST);
        frame.add(new JButton("Advertisement"), BorderLayout.EAST);
        frame.add(new JLabel("Main News Content", JLabel.CENTER),
BorderLayout.CENTER);
        frame.add(new JLabel("© 2025 Ma be oop parhne parh rhe",
JLabel.CENTER), BorderLayout.SOUTH);
        frame.setSize(400, 250);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:



3. GridLayout

Theory:

Divides the container into a **grid of equal cells**.
Each cell is the **same size**, regardless of content.

Real-world Scenario:

You're creating a **calculator UI**, where buttons are laid out in a grid format like numbers and operators in rows and columns.

Code:

```
import javax.swing.*;
import java.awt.*;

public class GridLayoutCalculator {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Calculator UI");
        frame.setLayout(new GridLayout(4, 4, 5, 5)); // rows, cols, hgap,
vgap
        String[] buttons = {
            "7", "8", "9", "/",
            "4", "5", "6", "*",
            "1", "2", "3", "-",
            "0", "+", "=", "C"
        };
    }
}
```

```

        "4", "5", "6", "*",
        "1", "2", "3", "-",
        "0", ".", "=", "+"
    };
    for (String text : buttons) {
        frame.add(new JButton(text));
    }
    frame.setSize(300, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```

Output:



4. BoxLayout

Theory:

Arranges components in a **single row (X_AXIS)** or **single column (Y_AXIS)**.

Offers better control for vertical or horizontal stacking.

Real-world Scenario:

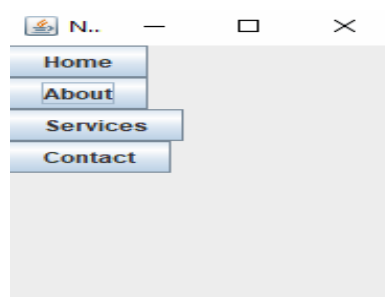
You are building a **vertical navigation menu** on a website with buttons like **Home**, **About**, **Services**, and **Contact**.

Code:

```
import javax.swing.*.*;
import java.awt.*.*;

public class BoxLayoutNavigationMenu {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Navigation Menu");
        JPanel panel = new JPanel();
        panel.setLayout(new BoxLayout(panel, BoxLayout.Y_AXIS));
        panel.add(new JButton("Home"));
        panel.add(new JButton("About"));
        panel.add(new JButton("Services"));
        panel.add(new JButton("Contact"));
        frame.add(panel);
        frame.setSize(200, 250);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setVisible(true);
    }
}
```

Output:



5. CardLayout

Theory:

Think of it as a **stack of cards**, where only one card (panel) is visible at a time.

Useful for building **wizard-style forms** or **step-by-step processes**.

Real-world Scenario:

You are creating a **multi-step registration form** with steps like:

Enter Personal Info

Choose Plan

Confirm Details

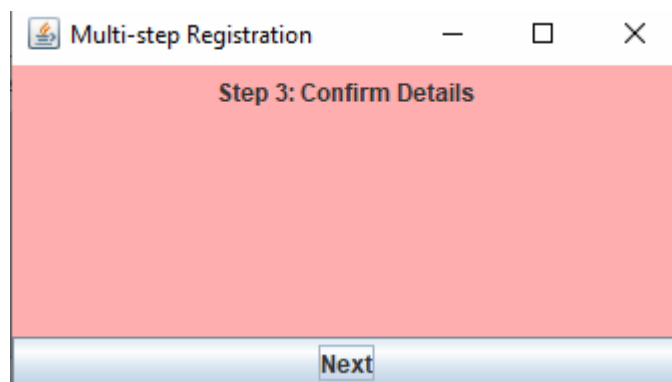
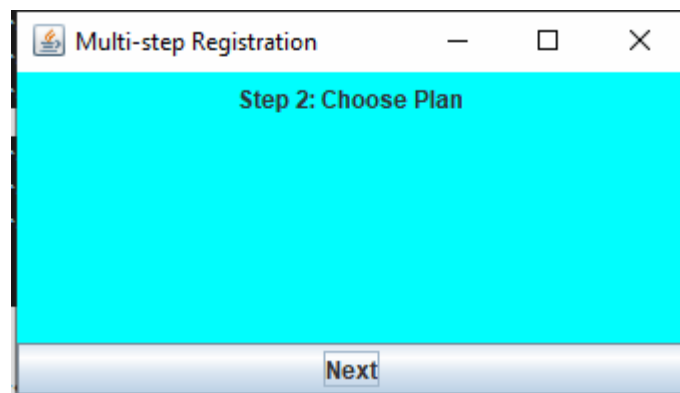
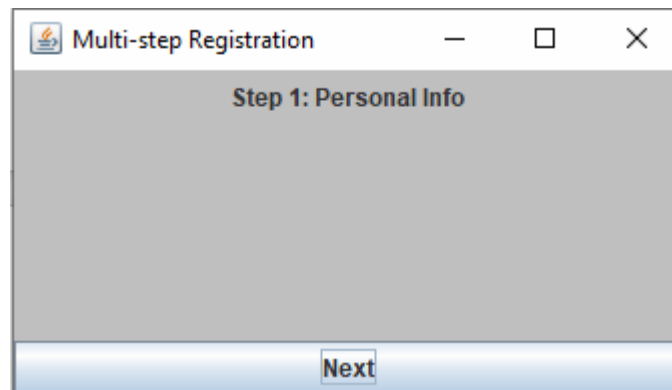
Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class CardLayoutRegistrationForm {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Multi-step Registration");
        CardLayout card = new CardLayout();
        JPanel cardPanel = new JPanel(card);
        JPanel step1 = new JPanel();
        step1.add(new JLabel("Step 1: Personal Info"));
        step1.setBackground(Color.LIGHT_GRAY);
        JPanel step2 = new JPanel();
        step2.add(new JLabel("Step 2: Choose Plan"));
        step2.setBackground(Color.CYAN);
        JPanel step3 = new JPanel();
        step3.add(new JLabel("Step 3: Confirm Details"));
        step3.setBackground(Color.PINK);
        cardPanel.add(step1, "1");
        cardPanel.add(step2, "2");
        cardPanel.add(step3, "3");
        JButton next = new JButton("Next");
        next.addActionListener(e -> card.next(cardPanel));
        frame.setLayout(new BorderLayout());
        frame.add(cardPanel, BorderLayout.CENTER);
    }
}
```

```
frame.add(next, BorderLayout.SOUTH);  
frame.setSize(350, 200);  
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
frame.setVisible(true);  
}  
}
```

Output:



Part 2: Event Handling in Java

Theory:

Event handling in Java is achieved through **Listeners**.

Most common: `ActionListener`, which listens to button clicks, menu selections, etc.

Real-world Scenario:

You are creating a **login form**, and you want the system to show a welcome message when the login button is clicked.

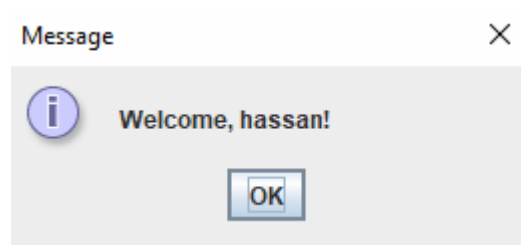
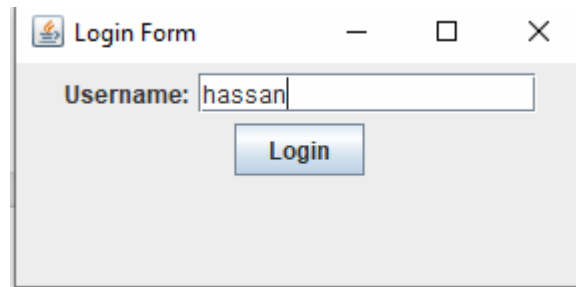
Code:

```
import javax.swing.*;
import java.awt.event.*;

public class LoginFormEventHandling {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Login Form");
        JTextField usernameField = new JTextField(15);
        JButton loginBtn = new JButton("Login");
        JPanel panel = new JPanel();
        panel.add(new JLabel("Username:"));
        panel.add(usernameField);
        panel.add(loginBtn);
        loginBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String user = usernameField.getText();
                if (!user.isEmpty()) {
                    JOptionPane.showMessageDialog(frame, "Welcome, " + user
+ "!");
                } else {
                    JOptionPane.showMessageDialog(frame, "Please enter a
username.");
                }
            }
        });
        frame.add(panel);
        frame.setSize(300, 150);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

```
frame.setVisible(true);  
}  
}
```

Output:



Navigating Between Two Screens (Windows) in Java Swing:

how to **navigate from one screen (JFrame)** to another, and **return back** to the previous screen using **event handling**.

In **Java Swing**, navigation between screens is typically done by:

- Creating multiple **JFrame**

- Hiding the current window and showing another one using

- setVisible(false/true)**.

- Using **ActionListener** on buttons to control navigation.



Real-World Scenario:

Imagine you're designing a **student portal**. On the **main dashboard**, the student clicks on "**Profile**", which opens a new screen displaying their details. They should be able to **click "Back"** to return to the main dashboard.

Code:

Implementation:

Step 1: MainDashboard.java (First screen)

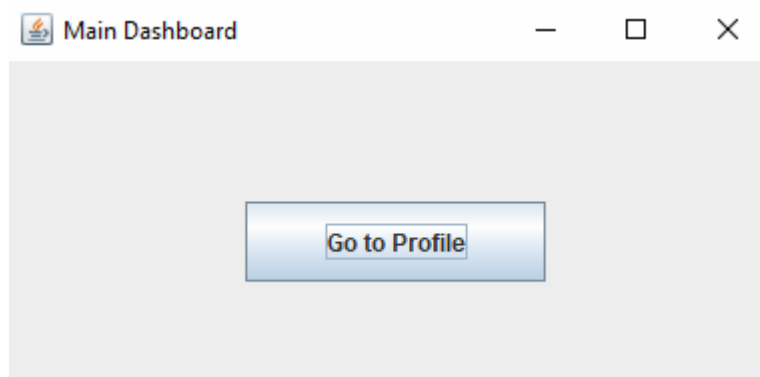
```
import javax.swing.*;
import java.awt.event.*;

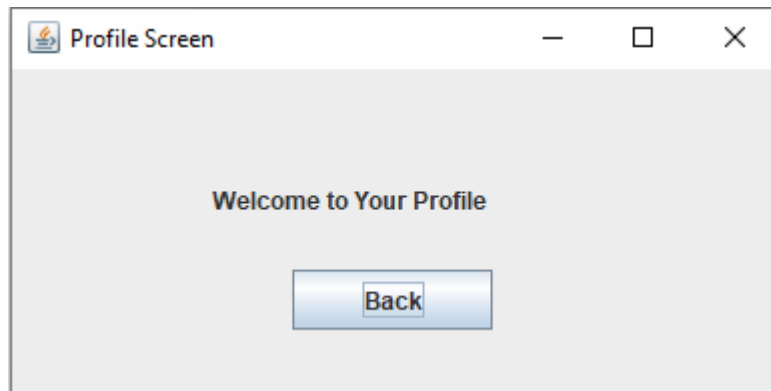
public class MainDashboard extends JFrame {
    JButton profileBtn;
    public MainDashboard() {
        setTitle("Main Dashboard");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);
        profileBtn = new JButton("Go to Profile");
        profileBtn.setBounds(120, 70, 150, 40);
        add(profileBtn);
        profileBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // Hide this screen and open profile
                setVisible(false);
                new ProfileScreen(MainDashboard.this);
            }
        });
        setVisible(true);
    }
    public static void main(String[] args) {
        new MainDashboard();
    }
}
```

Step 2: ProfileScreen.java (Second screen):

```
import javax.swing.*;
import java.awt.event.*;

public class ProfileScreen extends JFrame {
    JButton backBtn;
    MainDashboard mainDashboard; // reference to return
    public ProfileScreen(MainDashboard mainDashboard) {
        this.mainDashboard = mainDashboard;
        setTitle("Profile Screen");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(null);
        JLabel label = new JLabel("Welcome to Your Profile");
        label.setBounds(100, 50, 200, 30);
        add(label);
        backBtn = new JButton("Back");
        backBtn.setBounds(140, 100, 100, 30);
        add(backBtn);
        backBtn.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                // Close this screen and return to main dashboard
                dispose();
                mainDashboard.setVisible(true);
            }
        });
        setVisible(true);
    }
}
```





Full Concept in Simple Words:

Action	What You Do in Code
Open new screen	<code>new ProfileScreen();</code> or <code>new AnotherFrame();</code>
Hide current screen	<code>setVisible(false);</code>
Go back to previous	<code>dispose();</code> and then <code>previousFrame.setVisible(true);</code>
Store previous screen	Pass it like <code>new ProfileScreen(this);</code>

Types of Listener in java:

Listener Type	Common Usage	Trigger Example
ActionListener	Button clicks	JButton, JMenuItem
MouseListener	Mouse clicks, movement	Drawing apps, drag & drop
KeyListener	Keyboard input	Games, search bars
ItemListener	Dropdowns, checkboxes	Preferences, filters
FocusListener	Form fields	Input validation
WindowListener	Window control	Confirm on exit